

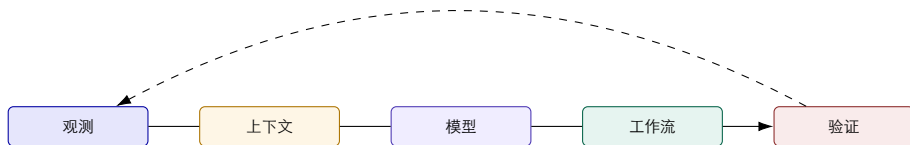
面向 AI 原生系统的智能运维

第 12 讲 | 大模型驱动的平台架构进阶

把模型、检索、工具、记忆、评测和安全接入 AIOps 控制面

陈鹏飞 | 中山大学

从第 11 讲继续：控制面里为什么要放大模型？



第 11 讲 平台统一对象、事件、状态、权限和证据。

本讲 模型把自然语言和异构证据接入控制面。

第 13 讲 再实现面向 RCA 的 Agent 开发和实验。

开场案例：答案很像，但为什么不能直接执行？

模型输出

- ▶ “可能是缓存命中率下降导致购物车超时”；
- ▶ 给出一条看起来合理重启命令；
- ▶ 语言流畅，解释完整，信心很高。

平台必须追问

- ▶ 这个结论引用了哪些指标、Trace 和变更？
- ▶ 数据是否新鲜，是否属于当前租户和集群？
- ▶ 动作会影响多少副本，能否回滚和验证？
- ▶ 如果证据冲突，系统如何停止而不是继续生成？

本讲主线 大模型不是平台的替代品，而是运行在对象、证据、状态、策略和评测之上的一个概率组件。

预算准入算例：两个 Agent 为什么会一起超支？

任务剩余预算 100 单位；两个并行分支各估计最坏花费 70。平台需在调用前原子预留预算。

操作	可用余额	已预留	平台决策
初始	100	0	等待申请
A 申请 70	30	70	接受 A
B 再申请 70	30	70	拒绝或等待 B
A 实际花费 45	55	0	结算 45，退回预留差额 25

$$\text{可用} + \text{预留} + \text{已结算} = 100$$

推导与判断 只让两个分支分别读取“余额 100”，会同时放行 140。该不变量依赖原子更新；超时调用若计费未知，不应提前释放预留。

算例使用假设数据。

范围与非目标

本讲覆盖

- ▶ 模型网关与推理运行时；
- ▶ RAG 数据平面与证据链；
- ▶ Agent Runtime、工具、记忆和策略；
- ▶ 可观测性、评测、成本和故障注入。

留给第 13 讲

- ▶ RCA Agent 的具体代码结构；
- ▶ 诊断工具实现和实验脚本；
- ▶ Prompt 迭代与模型微调细节；
- ▶ 完整生产部署和现场接入。

架构课先定义边界与接口约定，开发课再实现查询、诊断和结果检查。

2 学时路线图

1. 控制面变化：大模型引入了新的状态、成本和风险；
2. 模型服务：网关、路由、KV Cache、Prefill/Decode 和 Goodput；
3. RAG 数据面：检索、重排、证据、缓存和质量—延迟控制；
4. Agent Runtime：状态机、工具、MCP、记忆、预算和审批；
5. 观测评测：Langfuse/OTel、Token/GPU、Outcome、回放和注入；
6. 综合蓝图：把一次 AIOps 请求串成可验证的系统。

一、大模型进入平台后发生了什么

从确定性的查询和流程，进入概率性的模型、检索和多步执行

传统平台与 LLM 驱动平台的差异

维度	传统平台	LLM 驱动平台
输入	结构化事件、规则和表单	自然语言、半结构化证据和多轮上下文
状态	事件状态机和工单状态	任务、模型调用、检索、工具、记忆和审批状态
输出	查询结果、规则动作、固定模板	候选、解释、工具调用、计划和结构化结果
不确定性	规则命中或未命中	置信度、证据缺口、幻觉和模型漂移
成本	查询、存储和执行资源	Token、GPU、检索、工具、重试和人工接管

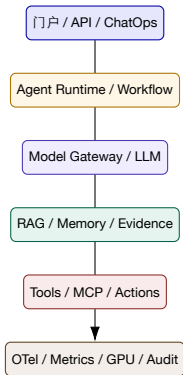
结论 LLM 不是在平台上新增一个“聊天入口”，而是新增一条需要观测和治理的概率执行链。

大模型在 AIOps 平台的四个插入点



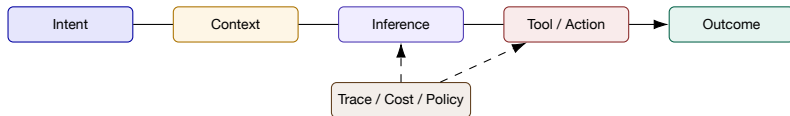
- ▶ 检测入口：把告警、工单和自然语言问题转成结构化意图；
- ▶ 检索入口：按对象、时间、权限和版本获取证据；
- ▶ 规划入口：选择工具、步骤、验证和人工接管；
- ▶ 解释入口：把候选、证据、缺口和下一步讲给人听。

六层架构：模型只是中间的一层



任何一层不可观测、不可审计或不可降级，都会削弱上层模型的可信度。

请求状态：一次问答其实是一个有生命周期的对象



- ▶ ‘intent’ 记录用户真正想完成的任务，而不是只记录 Prompt 文本；
- ▶ ‘context’ 记录检索、历史、权限和时间水位；
- ▶ ‘outcome’ 记录状态是否改变、验证是否通过和成本是否可接受。

模型网关的职责：把“调用模型”变成平台能力

路由 模型、版本、区域、租户和负载。

策略 预算、数据驻留、敏感信息和用途限制。

可靠性 超时、重试、熔断、降级和幂等。

观测 Token、延迟、模型、错误、质量和成本。

接口原则 应用不应直接依赖某个模型 SDK；模型网关提供稳定的请求、响应、错误和采集字段约定。

模型路由：不是“最强模型优先”

路由目标（简化模型）

$$m^* = \arg \min_m \text{Cost}(m) \quad \text{s.t.} \quad Q(m, x) \geq q_0, L_{p95}(m, x) \leq l_0, R(m) \leq r_0$$

- ▶ Q ：任务质量、引用正确性或工具选择正确性；
- ▶ L ：TTFT、TPOT、总时延和队列等待；
- ▶ R ：风险等级、数据合规、动作权限和故障影响；
- ▶ 质量、延迟、风险、Token 和 GPU 成本共同决定路由。

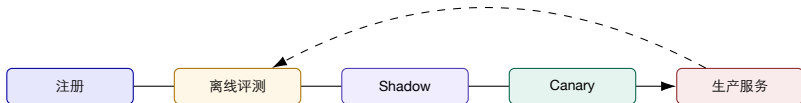
路由结果应写入 Trace，否则出现质量回归时无法解释“当时用了哪个模型”。

多模型路由案例： 同一个故障， 不同任务不同模型

任务	首选模型	备用模型	约束
告警摘要	小型低延迟模型	通用大模型	需引用事件和证据，不允许执行
复杂 RCA 候选 自然语言查询	推理能力强的模型 结构化输出模型	专用分类器 规则模板	需返回证据缺口和候选排序 查询范围、字段和租户严格限制
审批说明	通用模型 + 模板	人工撰写	不能替代审批人和安全策略

结论 模型选择是平台策略，不是 Prompt 中的个人偏好；路由策略必须有版本、评测和回滚。

模型生命周期：上线前后都是平台问题



- ▶ 评测集、Prompt、工具 schema 和 judge 版本一并登记；
- ▶ Shadow 只记录结果，不影响用户；Canary 才逐步承接流量；
- ▶ 质量、延迟、Token、成本和安全回归任一超门槛，都应自动回退。

Prompt 格式要求：把自由文本变成可审计输入

字段	示例	平台用途
任务类型	alert_summary、query、diagnosis	路由、预算和评测集
对象范围	tenant、service、cluster、incident_id	权限和上下文边界
证据引用	query_id、trace_id、knowledge_id	provenance、回放和解释
输出格式	JSON schema、候选列表、下一步	验证器和工作流
停止条件	缺失证据、超预算、风险升级	降级和人工接管

Prompt 是输入格式的一部分，但不能承担全部安全逻辑；权限和操作前检查必须在模型之外。

大模型平台的五条设计原则

1. 证据先于解释：模型只能解释已经收集并有来源的证据；
2. 结构先于文本：中间状态用 schema、事件和 Trace 表达；
3. 策略先于动作：模型提出建议，策略引擎决定是否允许；
4. 质量与成本并列：成功答案、Token、GPU 和人工接管一起评估；
5. 失败可回放：保存请求、上下文、模型、工具、状态和 Outcome。

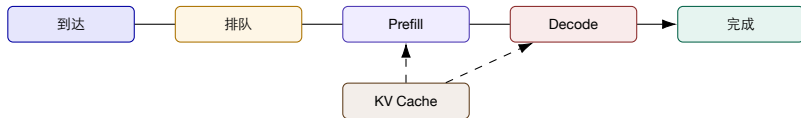
第一部分小结：怎样检查和约束模型给出的决策？

1. LLM 平台新增了模型、Prompt、检索、工具、记忆和多步状态；
2. 模型网关负责路由、策略、可靠性、观测和生命周期；
3. 质量、延迟、成本、风险和证据缺口必须成为同一请求的属性；
4. 下一部分从网关下钻到推理运行时，解释模型服务为什么会慢、贵或不稳定。

二、模型网关与推理运行时

模型服务不是黑盒 API: Prefill、Decode、KV Cache、调度和 GPU
共同决定体验

一次生成请求的运行时生命周期



- ▶ 排队决定用户等多久；Prefill 决定首 Token；Decode 决定流式速度；
- ▶ KV Cache 连接 Prompt 长度、并发、显存和请求抢占；
- ▶ 同样的 E2E 延迟，可能来自完全不同的阶段瓶颈。

TTFT、TPOT 和 Goodput：运行时指标如何对应用户？

TTFT

请求到首 Token

平均 TPOT

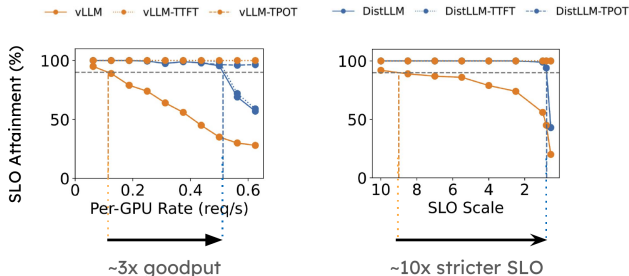
首 Token 后每输出 Token 的平均时间；
单次间隔看 ITL

Goodput

满足 SLO 的完成请求/时间

- ▶ TTFT 变慢：先查排队、路由、Prefill 和 Prompt 长度；
- ▶ TPOT 变慢：先查 Decode、Batch、KV、GPU 计算和通信；
- ▶ Goodput 下降：不仅是吞吐下降，也可能是质量、超时或 SLO 违约。

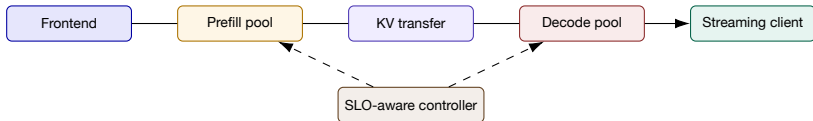
OSDI 2024 DistServe: 为什么要解耦 Prefill 与 Decode?



论文观察

- ▶ Prefill 偏计算密集，Decode 更受内存带宽和逐Token 交互影响；
- ▶ 混部会让不同阶段互相干扰；
- ▶ 解耦后可按 TTFT 与 TPOT 的约束分别分配资源；
- ▶ 评价重点从 requests/s 转为 SLO attainment 和 Goodput。

DistServe 的系统机制：阶段独立，不等于网络开销消失



- ▶ KV 传输、路由和队列是解耦架构新增的观测点；
- ▶ 资源比例不能固定，需要随输入长度、输出长度和负载变化调整；
- ▶ 结论不是“解耦永远更快”，而是“在 SLO 约束下更容易管理阶段干扰”。

DistServe 的结果应该怎样读？

SLO attainment

负载提高时仍能保持更多请求满足
约束

Goodput

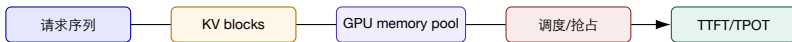
完成且可用的请求才计入收益

阶段可控

TTFT/TPOT 可以分别诊断与扩容

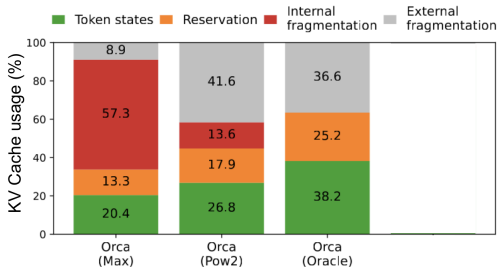
- ▶ 论文图中严格 SLO 和高负载下的架构差异更明显；
- ▶ 结果依赖模型、GPU、输入输出长度、并发和网络配置；
- ▶ 平台结论：容量看板必须同时展示阶段延迟、SLO attainment、Goodput 和 KV 传输。

PagedAttention: 显存管理是服务质量问题



- ▶ 固定连续内存会造成预留、碎片和请求间浪费；
- ▶ 分页式 KV 管理按块分配，支持共享、动态增长和更高并发；
- ▶ 诊断时要区分“物理显存不足”“KV block 不足”“调度等待”和“计算瓶颈”。

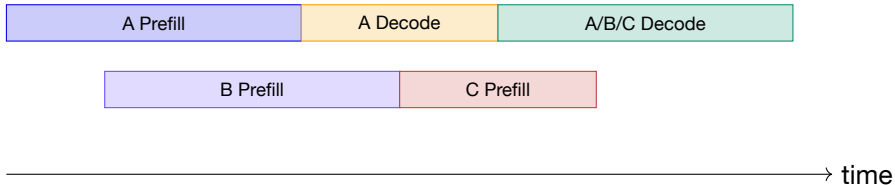
KV Cache 的结果：容量、效率和压力必须一起看



- ▶ 物理显存占用高，不代表有效 Token 状态比例高；
- ▶ 长 Prompt、长输出和并发请求共同决定 KV 压力；
- ▶ Prefix Cache 命中可以减少 Prefill 工作，但会增加缓存管理和一致性约束；
- ▶ 结论：KV 指标必须关联请求 Trace、Prompt 长度、命中率和抢占事件。

连续批处理：调度器决定“谁先得到 GPU 时间”

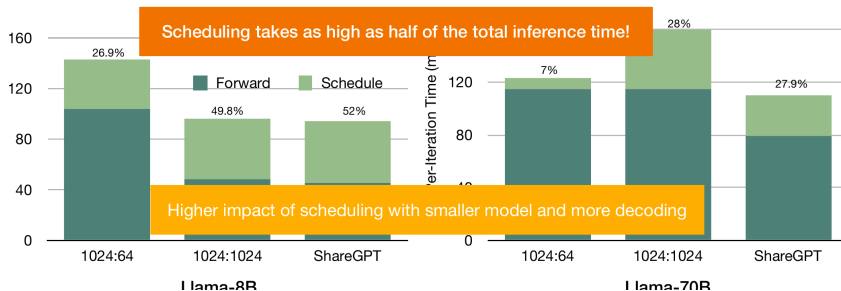
请求进入



- ▶ 连续批处理提高设备利用率，但会改变每个请求的等待和 Decode 机会；
- ▶ 调度器需要同时看队列年龄、Token 预算、KV 资源和 SLO；
- ▶ 结果验证不能只看平均吞吐，还要看长尾和饥饿请求。

调度开销也是平台故障的一种表现

- Llama-8B and Llama-70B running synthetic and real workloads on RunPod A100 GPUs



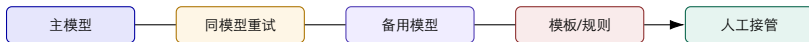
症状 GPU 利用率并不低，但 TTFT/TPOT 长尾恶化。

候选 Batch 变化、调度等待、KV 抢占、请求长度和路由重试。

模型运行时的故障分类

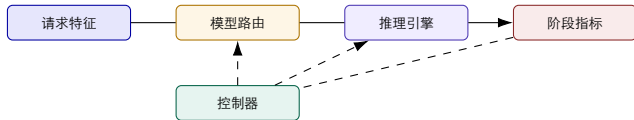
故障层	表面症状	需要的证据	常见误判
网关 队列	超时、重试、错误码 TTFT 上升、长尾	route、model、region、quota arrival、queue age、batch、priority	直接归因于模型质量 认为 GPU 不够
Prefill	首 Token 慢	prompt length、KV、compute、 cache	认为 Decode 慢
Decode	Token 间隔变大	batch、memory bandwidth、 output length	只看平均吞吐
GPU/网络	降频、通信抖动、错误	DCGM、kernel、NCCL、XID、 Trace	把末端症状当根因

模型网关的降级策略



- ▶ 重试必须考虑幂等、成本和错误是否可恢复；
- ▶ 备用模型要重新评估输出 schema、质量和工具权限；
- ▶ 模板模式只承诺可验证的固定能力，不伪装成自由生成；
- ▶ 关键事件在模型和模板都不可信时，直接进入人工接管。

运行时控制环：资源状态反馈到路由和调度



- ▶ 请求特征：Prompt 长度、输出预算、任务类型和优先级；
- ▶ 资源状态：队列、KV、GPU、网络、错误、SLO attainment 和成本；
- ▶ 控制器可以调整模型、批大小、并行度、缓存和降级，但必须有边界。

模型服务实验：必须同时报告结果和结论

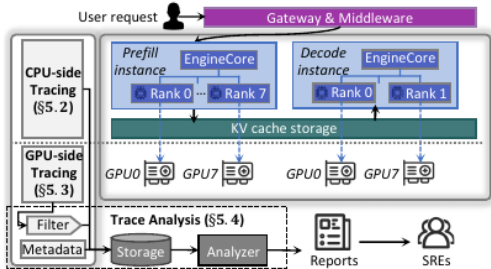
实验项	结果应该报告什么	不能直接下的结论
路由 调度 KV/缓存 P/D 解耦	质量、P95 TTFT/TPOT、Token、成本和降级率 Goodput、长尾、饥饿、GPU 利用和队列年龄 命中率、有效 Token、显存、抢占和重算 SLO attainment、KV 传输、资源比例和网络开销	“大模型一定更好” “吞吐更高就更稳定” “缓存高就是系统健康” “解耦在所有负载都更快”

证据要求 每个算法或系统机制都必须有可观测结果、适用条件和边界结论。

第二部分小结：运行时必须按阶段诊断

1. 网关负责路由、策略、生命周期和降级；
2. Prefill、Decode、KV、调度、GPU 和网络共同决定用户体验；
3. DistServe 的核心启发是按 SLO 管理阶段干扰；PagedAttention 的核心启发是按块管理 KV；
4. 运行时结果要看 Goodput、长尾、资源和成本，而非单一吞吐数字；
5. 下一部分进入 RAG 数据平面：模型为何“有证据却答错”或“答得快却不可信”。

OSDI 2026 StriaTrace: 在线推理为何不能全函数 tracing?



- ▶ vLLM EngineCore 单步可有 10,000+ 次函数调用;
- ▶ 只追跨进程 RPC、外部同步点和语义关键路径;
- ▶ 通过 request/instance ID 连接 prefill、decode、KV 与 GPU rank;
- ▶ 平时低开销, 异常时才提高 GPU 侧细节。

StriaTrace 的诊断结果：从 SLO spike 回到关键路径

97.8%

相对替代方案的 tracing overhead
降低

19

公开报告的根本原因类型

数百

开发、测试与生产异常

- ▶ 动态 regression-based roofline 用于发现性能偏离，相关诊断回到阶段/rank/kernel；
- ▶ “97.8% 降低”是相对量，不代表绝对 overhead 为 2.2%；
- ▶ 关键同步点与语义 span 依赖 runtime 架构，换引擎需要重新映射；
- ▶ 第 17 讲将进一步回答 Agent 何时调用这类工具、如何用结果更新竞争假设。

三、RAG 与证据数据平面

让模型先找到适用证据，再生成可验证解释

RAG 不是“向量库 + Prompt”



- ▶ 召回解决“找得到”，重排解决“排在前”，证据约束解决“用得对”；
- ▶ 生成器必须看到来源、版本、适用范围和冲突信息；
- ▶ 验证器检查引用、覆盖、事实一致性和最终状态。

RAG 数据格式约定：每个 Chunk 都要带上下文

字段	示例	作用
身份	doc_id、chunk_id、version	去重、回放、更新和引用
范围	tenant、service、environment	权限和检索过滤
时间	valid_from、valid_to、updated_at	处理过期知识和变更窗口
来源	URL、ticket、runbook、owner	provenance 和人工复核
语义	title、section、tags、embedding	关键词、向量和混合检索
质量	reviewed、confidence、deprecated	过滤低质量和冲突内容

没有版本和适用范围的 Chunk，检索命中率越高，过期知识传播得越快。

切分策略：短 **Chunk** 不一定更好

- ▶ 过短：语义不完整、引用缺少前置条件；
- ▶ 过长：召回噪声增加、上下文预算被占用；
- ▶ 按标题、步骤、表格、代码块和故障模式切分；
- ▶ 保留父文档、邻接 **Chunk** 和结构路径。

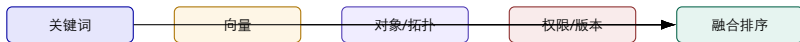
切分评测

$$\text{有效证据率} = \frac{\text{\#被答案正确使用的片段}}{\text{\#召回片段}}$$

$$\text{上下文利用率} = \frac{\text{答案引用的证据 Token}}{\text{输入上下文 Token}}$$

结论 切分参数要用真实问题和引用结果评测，而不是只看向量相似度。

混合检索：关键词、向量和结构过滤各自解决什么？



- ▶ 关键词擅长错误码、服务名、版本和精确术语；
- ▶ 向量擅长语义相近和自然语言描述；
- ▶ 图和结构过滤把对象、拓扑、租户和版本边界前置；
- ▶ 融合排序要保留每个召回通道的分数和来源。

Rerank: 把“相关”改成“适用于当前事件”

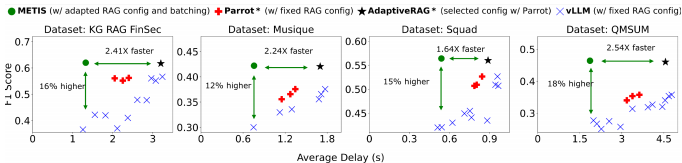
重排分数（简化模型）

$$S(d, q) = w_v V(d, q) + w_s S(d, q) + w_t T(d) + w_o O(d, q) - w_x X(d)$$

- ▶ V: 向量/关键词相关性;
- ▶ S: 服务、环境、租户和事件类型匹配;
- ▶ T: 版本和时间有效性;
- ▶ O: owner、复审状态、历史成功使用;
- ▶ X: 冲突、过期与缺少前置条件; 租户和权限先做硬过滤。

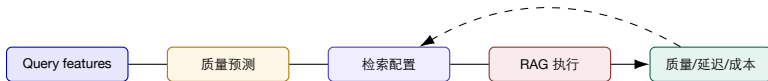
排序只在有权访问且适用的候选内进行; 高相关分不能抵消越权、失效版本等禁止条件。

METIS: RAG 配置应该随 Query 和资源状态变化



- ▶ 不同 Query 对召回数量、重排和上下文长度的收益不同；
- ▶ 同一配置在质量、延迟和成本上可能相互冲突；
- ▶ Query 特征、资源状态和质量预测应进入控制环；
- ▶ 结论：不存在对所有 Query 都最优的固定 RAG 配置。

RAG 控制环：质量、延迟和成本如何一起优化？



- ▶ 低风险 Query 可以使用轻量召回和短上下文；
- ▶ 高风险诊断 Query 提高证据覆盖、重排和人工复核；
- ▶ 资源拥塞时可以降级检索深度，但必须显式降低置信度。

METIS 类方法的结果与结论

Query-aware

同一配置不再服务所有问题

Quality-latency

把质量和延迟放进同一控制环

可降级

资源压力时保留证据缺口

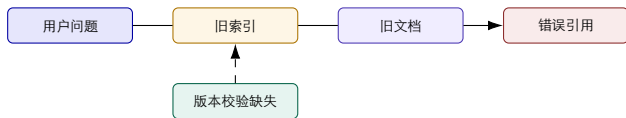
- ▶ 论文/材料中的提升来自 Query 特征、质量预测和配置选择的联合；
- ▶ 若质量标签延迟、预测误差大或 Query 分布漂移，控制环会失效；
- ▶ 平台应报告每种配置的覆盖、延迟、质量、成本和降级率。

RAG 质量矩阵：找得到、用得好、答得对

层级	典型指标	失败例子
召回 证据 生成 任务 运营	Recall@k、MRR、coverage、freshness precision、groundedness、citation correctness factuality、completeness、format validity success、state change、human acceptance latency、cost、fallback、feedback	相关文档没有召回，或只召回旧版本 文档被召回，但答案没有使用对应句子 引用存在但结论超出证据 文本看似正确，但没有完成查询/处置 质量提升却超预算或频繁接管

结论 RAG 的最终指标应落在任务 Outcome，而不是只停留在向量检索分数。

案例：RAG 延迟正常，但答案引用了旧政策

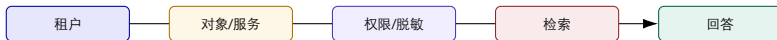


- ▶ 延迟、GPU 和 Token 都可能正常；
- ▶ 真正异常在知识版本、有效期和引用校验；
- ▶ LLMRCA 把 context 节点纳入诊断图，避免只盯着 generator；
- ▶ 修复动作是更新索引、撤销旧文档和重新验证，不是盲目换模型。

RAG 缓存：命中率高也可能带来错误

- ▶ Query cache: 问题相同或语义相近时复用答案;
- ▶ Retrieval cache: 复用候选文档和重排结果;
- ▶ Embedding cache: 减少重复向量计算;
- ▶ Prefix cache: 减少模型 Prefill 工作。
- ▶ TTL、版本、租户和权限必须进入缓存键;
- ▶ 发生配置、知识或权限变化时主动失效;
- ▶ 缓存命中需要和 groundedness、freshness、成本一起看;
- ▶ 缓存绕过后的结果也要保留对照 Trace。

多租户 RAG：同一个问题不一定能看同一批文档



- ▶ 权限过滤要在召回前和生成前都执行；
- ▶ 文档的租户、服务、环境和数据驻留标签不能被模型自行推断；
- ▶ 证据包必须记录被过滤的范围和缺失原因，避免“看不到”被误解成“不存在”。

RAG 失败模式：从数据到答案逐层排查

阶段	失败模式	验证动作
摄取	文档漏采、解析错误、版本未更新	对象覆盖、版本 diff、摄取延迟
切分	关键前置条件被拆开	片段上下文、父子关系、人工抽查
召回	相关文档未命中或旧文档优先	Recall@k、freshness、query replay
重排	服务/租户/版本不匹配	过滤日志、分数解释、权限测试
生成	超出证据、引用错位或格式错误	citation check、schema validator、事实核验
任务	没有完成状态改变或验证	Outcome oracle、状态对照、人工确认

RAG 的 Evidence Bundle：把证据交给模型，也交给评测

证据包（简化模型）

`query_id`、`tenant`、`object_scope`、`retrieval_config`、`chunks[]`、`versions[]`、`citations[]`、`missing[]`、`quality_scores`。

- ▶ 模型只消费带 `provenance` 的证据，而不是直接访问未治理数据库；
- ▶ 评测可以复现同一 Query 的召回、重排和生成条件；
- ▶ 发生质量回归时，能够区分索引、配置、模型和 Prompt 的变化。

RAG 实验：结果和结论必须分开写

实验结果	可以支持的结论	不能支持的结论
Recall@k 上升 Groundedness 上升 P95 延迟下降 成本下降 人工接受率上升	相关证据更容易被找到 生成更常引用召回证据 当前 Query 分布下响应更快 当前配置减少 Token/检索/GPU 该人群和任务下更愿意采用	答案一定更正确 证据本身一定真实或最新 所有 Query 都更快 任务成功率没有损失 可以无审批自动执行

研究生要求 每个结果都要附数据集、Query 分布、配置、基线、置信区间或误差来源。

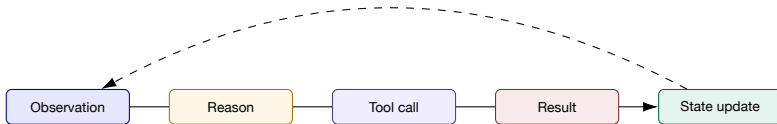
RAG 数据平面的小结

1. RAG 包括检索、重排、生成答案和检查引用是否支持答案；
2. METIS 的启发是 Query-aware 的质量—延迟控制；
3. LLMRCA 的启发是把知识版本和上下文质量纳入故障诊断；
4. 缓存、多租户、版本和权限决定证据是否仍然适用；
5. 下一部分进入 Agent Runtime：当模型不只回答，还会规划和调用工具。

四、Agent Runtime 与控制面

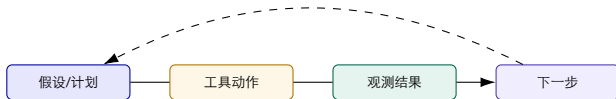
让模型的思考、工具、状态、记忆和动作进入可恢复、可审计的工作流

Agent 运行时：保存状态、调用工具并检查结果



- ▶ Agent Runtime 保存循环，不把状态藏在 Prompt 文本中；
- ▶ 每次模型调用和工具调用都要有输入、输出、错误、耗时和权限；
- ▶ 状态更新需要验证器，避免模型自行宣布“已经修复”。

ReAct: Reason + Act + Observation



- ▶ 优点：模型可根据新证据动态调整调查顺序；
- ▶ 风险：循环、重复查询、状态漂移、无限重试和成本失控；
- ▶ 工程补充：最大步数、预算、状态机、超时、验证和人工接管。

Plan-and-Execute 与 StateFlow：把多步任务显式化

Plan-and-Execute

- ▶ 先生成步骤，再逐步执行；
- ▶ 易做覆盖检查和阶段验收；
- ▶ 计划过时后需要重新规划。

StateFlow

- ▶ 用状态、转换和守护条件表达任务；
- ▶ 可恢复、可暂停、可回放；
- ▶ 复杂分支需要更清晰的状态设计。

平台结论 自由循环适合探索，显式状态适合生产；企业平台要保存两者之间的“为什么转换”。

Agent 状态模型：把隐含信息写成结构化字段

字段	示例	失败时怎么用
任务	task_id、goal、deadline	判断是否仍然属于原任务
假设	candidate、support、counterevidence	防止结论升级过快
步骤	step_id、parent、status、retry	回放、重试和断点恢复
工具	tool、args、scope、result	审计、幂等和错误分类
记忆	memory_id、source、validity	区分历史经验与当前证据
终态	outcome、validator、human_review	判断是否真的完成

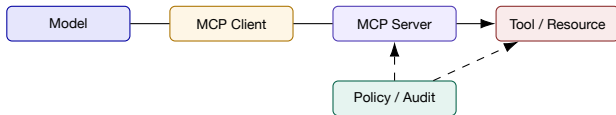
工具接口约定：从“能调用”到“可验证”

工具接口说明（简化模型）

name、version、scope、input_schema、output_schema、error_semantics、timeout、idempotency、risk、rollback。

- ▶ 查询工具返回数据水位、来源和缺失字段；
- ▶ 执行工具返回前后状态、结果、错误和副作用；
- ▶ 模型看到的是工具接口说明和摘要，不是未经治理的任意 API；
- ▶ 接口约定变更必须触发回放、兼容测试和权限复核。

MCP：连接模型与工具，但不等于安全边界



- ▶ MCP 解决工具和资源的标准化发现与调用接口；
- ▶ 租户、资源作用域、审批、数据脱敏和动作风险仍由平台策略负责；
- ▶ 不能因为工具“可发现”就默认允许模型执行。

工具注册表：模型选择工具，平台选择风险

- ▶ 工具用途、输入、输出和错误；
- ▶ 支持的租户、集群、服务和数据范围；
- ▶ 风险等级、审批要求和回滚方式；
- ▶ 延迟、成功率、成本和历史案例。
- ▶ 只读工具可自动调用，但仍要审计；
- ▶ 建议工具生成候选动作和前置检查；
- ▶ 执行工具必须经过策略、审批和结果验证；
- ▶ 低质量工具要降权、隔离或下线。

结论 Agent 的“智能”来自上下文和工具组合；平台的“可靠”来自接口约定、权限和验证。

记忆分层：上下文越多，不代表诊断越好



- ▶ 工作记忆：当前事件、计划、工具结果和未决假设；
- ▶ 事件记忆：历史故障、时间线、证据和动作结果；
- ▶ 语义记忆：服务知识、Runbook、约束和术语；
- ▶ 策略记忆：权限、禁用动作、阶段验收和验证规则；
- ▶ 每层都要有来源、有效期、owner 和冲突处理。

上下文预算：Token 是可靠性约束，不只是成本

上下文预算（组成项）

$$B = B_{system} + B_{task} + B_{evidence} + B_{history} + B_{output}$$

- ▶ 证据过少：模型无法解释或提出错误假设；
- ▶ 历史过多：最新事件被旧结论淹没；
- ▶ 输出预算过小：计划或结构化结果被截断；
- ▶ Token 过多：Prefill 慢、成本高、上下文注意力分散；
- ▶ 平台应按任务类型、风险和验证需要动态分配预算。

模型一Agent 编排：为什么需要多个专门角色？

端到端训练管线



vLLM rollout + FSDP2 训练 · 异步分布式管线

- ▶ **Planner**: 拆分任务和选择证据路径;
- ▶ **Investigator**: 调用指标、日志、Trace 和 CMDB;
- ▶ **Critic/Judge**: 检查时间、拓扑、反证和置信度;
- ▶ **Executor**: 只在策略允许时提出或执行动作;
- ▶ **Recorder**: 把轨迹、结果和反馈写回评测与知识。

决策链：模型建议、平台验证、人决定动作



一次任务的处理顺序

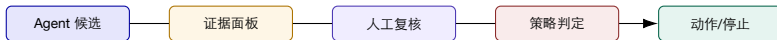
Intent → Evidence → Hypothesis → Check → Decision → Action → Verify。

多智能体的协调失败：拆能力，不拆责任

失败模式	具体表现	平台约束
重复调查 证据冲突 循环转交 责任漂移	Metric Agent 和 Log Agent 查同一窗口 不同 Agent 对异常定义不一致 每个 Agent 都要求另一个确认 结论无人签字，动作无人负责	共享任务队列、结果缓存和预算 统一 schema、时间水位和置信度 明确汇总 Agent、终态和升级路径 owner、审批人、接管人和审计

多 Agent 的价值来自不同视角和可合并证据，不来自 Agent 数量。

Human-in-the-loop: 人不是最后按一下按钮



- ▶ 人工需要看到候选、证据、缺口、风险和预期副作用；
- ▶ 审批结果成为事件状态和评测标签，不只是按钮点击；
- ▶ 高风险动作要求双人审批、回滚演练和执行后验证。

模型输出之外，还要做哪四类检查？

1. 输入门：租户、对象、敏感字段、Prompt 注入和数据范围；
2. 输出门：JSON schema、引用、敏感内容、置信度和拒答；
3. 工具门：权限、风险、参数、幂等、超时和目标校验；
4. 结果门：技术指标、业务 SLO、状态差异、回滚和人工确认。

结论 用独立程序检查权限、参数、预算和结果；不能只在 Prompt 中要求模型遵守。

动作策略：建议和执行必须有不同的状态



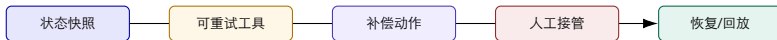
- ▶ 模型只能产生 Suggested;
- ▶ 审批人和策略引擎共同产生 Approved;
- ▶ Executed 只表示命令发出, Verified 才表示目标状态满足;
- ▶ 失败、超时和副作用都必须进入状态机。

预算与停止条件：Agent 不能无限思考

预算维度	典型信号	超限动作
步数	tool calls、state transitions	停止并交给人工/模板
Token	input/output、重试、历史膨胀	压缩上下文或切换模型
时间	wall time、queue、tool timeout	取消、保存断点、异步处理
成本	model、GPU、检索、执行和人工	降级、限流、审批
风险	目标范围、动作等级、影响面	阻断并要求复核

预算超限不是异常日志，而是任务状态的一部分；平台要告诉用户“为什么停止”。

Agent 失败恢复：从断点继续，而不是从 Prompt 重来



- ▶ 保存已完成步骤、工具结果、证据和未决假设；
- ▶ 只对幂等、低风险和可确认失败重试；
- ▶ 不可补偿动作进入人工接管，不能自动“再试一次”；
- ▶ 回放模式不触碰生产，验证通过后再恢复任务。

Agent 运行时的结果指标

Task pass

终态满足任务目标

Evidence

证据覆盖和引用正确

Safety

违规动作和误执行

Efficiency

步骤、Token、时间和成本

- ▶ pass@1 只说明一次任务成功，不解释为什么成功或失败；
- ▶ Step coverage、tool success、state validity 和 human acceptance 解释中间过程；
- ▶ 每个指标都要按任务类型、模型、工具版本和数据分段。

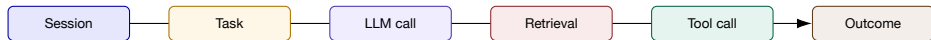
第四部分小结：Agent Runtime 是控制面上的状态机

1. ReAct 提供探索式循环，StateFlow/工作流提供生产可恢复性；
2. MCP 和工具注册表解决发现与接口约定，策略引擎解决权限与风险；
3. 记忆、上下文和预算是可靠性约束，不只是 Prompt 技巧；
4. 人工接管要看到证据、缺口、风险和预期结果；
5. 下一部分把这些运行时行为变成可查询的 Trace、评测和故障注入。

五、可观测性、评测与故障注入

没有 Trace、Outcome 和回放，LLM 平台只能被“感觉”驱动

LLM/Agent Trace：一条请求要跨越哪些粒度？



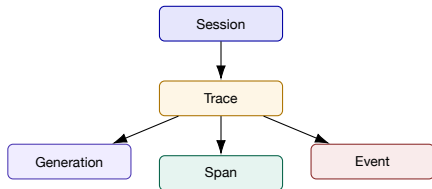
- ▶ Session：用户或业务会话；Task：一次可验收任务；
- ▶ LLM call：模型、版本、Prompt、Token、延迟、Finish reason；
- ▶ Retrieval/Tool：查询、文档、参数、结果、错误和权限；
- ▶ Outcome：状态变化、业务 SLO、人工接受、回滚和成本。

OpenTelemetry GenAI: 统一语义仍然需要适配层

观测对象	关键属性	诊断问题
模型调用	provider、model、request、finish reason、input/output tokens	哪个模型、哪类输入、哪种结束原因？
检索	query、top-k、index、documents、scores	找到了什么，是否最新和适用？
工具	name、args、scope、status、latency、error	工具是否被正确调用，是否越权？
Agent 步骤	step、parent、state diff、budget	错误从哪一步开始，是否循环？
结果	score、feedback、validator、human review	任务真的完成了吗？

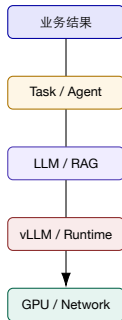
属性名称随规范和 SDK 演进，内部平台应提供版本化适配层，避免把某个后端字段当成永久标准。

Langfuse 类数据模型：Observation、Trace、Session



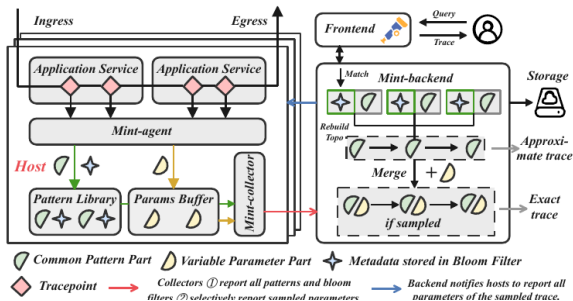
- ▶ Trace 把一次任务串起来；Observation 记录模型、检索、工具和验证步骤；
- ▶ Score、成本和人工反馈挂在 Trace 或 Observation 上；
- ▶ Langfuse/Tempo/Prometheus/DCGM 分工不同，但应共享 Trace ID 和对象身份。

把 Token、GPU 和业务 Outcome 放进一条链



- ▶ Outcome: 任务是否完成、引用是否正确、状态是否恢复;
- ▶ Task: 步骤、工具次数、重试、人工接管和预算;
- ▶ Model: Token、模型版本、Prompt、检索和响应质量;
- ▶ Runtime/GPU: 队列、TTFT、TPOT、KV、显存、通信和错误。

Mint: Trace 成本和诊断覆盖可以同时优化



- ▶ 共性结构低成本保留；差异参数按需保留；
- ▶ 未采样请求也能返回 approximate path；
- ▶ 论文报告平均存储约降至 2.7%，网络约降至 4.2%；
- ▶ 结论：Trace 采样不应只在“完整”和“丢弃”之间二选一。

Mint 的结果与边界：近似 Trace 也要有诚实标签

27.17%

传统采样查询 miss 的论文报告

2.7%

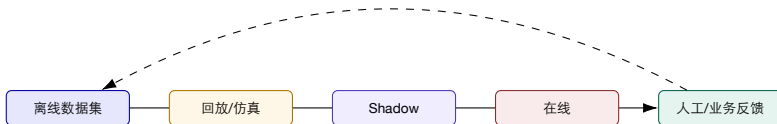
Mint 平均存储占完整追踪基线

4.2%

Mint 平均网络占完整追踪基线

- ▶ Approximate Trace 可以回答拓扑和路径问题，但不应伪装成精确参数；
- ▶ 频繁变化的 Trace 结构、节点故障、迟到和 Bloom Filter 假阳性会影响重建；
- ▶ 平台要返回 ‘exact’、‘partial’、‘missing’ 和重建版本，供算法和人判断。

评测体系：离线、回放、在线和人工反馈



- ▶ 离线：固定问题、证据、标签和基线；
- ▶ 回放：复现时间、工具、模型、数据和策略；
- ▶ Shadow：记录结果但不影响生产；
- ▶ 在线：观察分布、长尾、成本和人工接管；
- ▶ 反馈：把真实 Outcome 回填到数据集和版本治理。

LLM-as-Judge：有用，但不是客观真值机器

- ▶ 适合大规模初筛：相关性、完整性、风格、引用一致；
- ▶ 必须固定模型、Prompt、评分量规和输出 schema；
- ▶ 需要人工样本校准偏差和一致性；
- ▶ 对高风险动作不能只依赖 Judge 分数。

校准指标（示例）

$$\text{agreement} = \frac{\text{\#Judge 与人工一致样本}}{\text{\#校准样本}}$$

$$\text{calibration error} = |\rho_{\text{judge}} - \rho_{\text{human}}|$$

Judge 是测量工具，必须记录版本、提示词、样本和误差；不能把它当成事实本身。

评测结果：把数字连到结论

结果	可以支持的结论	仍需补充的证据
Task pass@1 上升 Groundedness 上升 MTTR 下降 Token/成本下降 人工接受率上升	该任务集合中一次成功率提升 答案更常由检索证据支持 受控场景恢复更快 当前模型和配置更省 使用者更愿意采用	多次重试、成本、失败类型和终态质量 文档是否真实、最新和适用 事故类型、人工接管和副作用 质量、长尾和错误重试 是否出现绕过、误信和责任转移

结果解读 解读结果时需明确指标定义、实验条件、结论强度，以及结论能否适用于其他环境。

TOSEM 2025 LLMRCA: LLM 应用的故障图要包含质量上下文

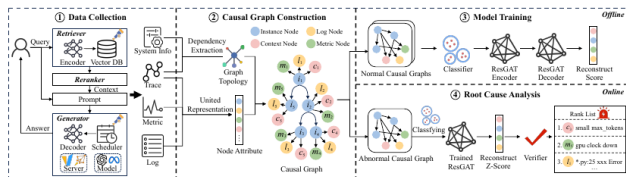


Fig. 7. The framework of LLMRCA.

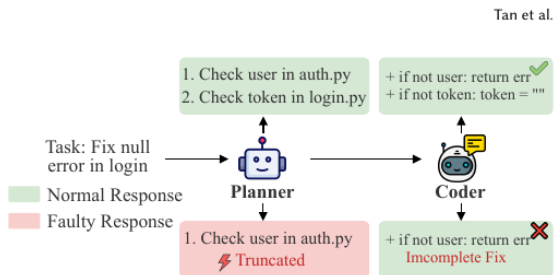
- ▶ instance、metric、log、context 形成异构诊断图；
- ▶ AP 关注性能故障，RQ 关注响应质量故障；
- ▶ verifier、residual 和 classifier 分别约束候选；
- ▶ 结果应返回组件、内部证据、质量根因和反证。

LLMRCA 的结果：三类任务分别读

任务	HR@1	NDCG@3	结果解释
AP component-level	84.23%	89.20%	性能故障组件定位
AP inner-component	73.81%	81.99%	组件内指标/日志/上下文定位
RQ root cause	92.86%	97.36%	响应质量故障定位

- ▶ 论文在一个 RAG 应用上注入 42 个故障，覆盖 10 类；
- ▶ 结果受故障分布、标签粒度和观测覆盖影响；
- ▶ 消融显示 verifier、residual 和 classifier 对不同任务的贡献不同；
- ▶ 结论：质量故障需要自己的观测节点，不能只看服务延迟。

AgentChaos: 故障注入和可诊断性一起设计



- ▶ 故障可能发生在 LLM API、Planner、Tool、State 或最终响应；
- ▶ 共享 HTTP 层注入可以不修改目标 Agent 源码；
- ▶ 结果不仅看 pass@1 下降，还要看能否定位故障类型和步骤；
- ▶ 诊断 Trace 必须保留 task、step、call、tool 和终态 oracle。

AgentChaos 的结果：最危险的是“坏了但诊不出”

50 pp

pass@1 最大下降

<53%

故障类型诊断

<56%

故障步骤诊断

- ▶ 所测系统在注入下均出现任务性能退化；
- ▶ 故障类型和故障步骤的诊断能力显著低于任务成功率；
- ▶ 结论：Agent 可靠性不能只测“能不能完成”，还要测“能不能解释为什么失败”；
- ▶ 数字依赖论文的系统、任务、模型和故障配置，不外推为通用生产保证。

可观测性结果的统一结论：从“有 **Trace**”到“能诊断”

1. **Trace** 覆盖：请求、模型、检索、工具、运行时和 **GPU** 是否连得起来？
2. **Trace** 质量：是否有 **parent**、时间、对象、版本、来源和数据水位？
3. **Trace** 可用性：未采样、迟到、缺失和重建结果是否有诚实标签？
4. **Trace** 结果：能否支持候选排序、回放、评测、审计和动作验证？

平台判据 观测系统的结果不是“采集了多少 **Span**”，而是一次失败能否更快、更安全、更可解释地被处理。

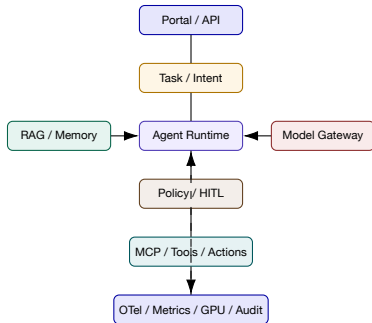
第五部分小结：评测和故障注入是控制环的一半

1. Langfuse/OTel 把模型、检索、工具、Token、成本和 Outcome 关联起来；
2. Mint 说明在成本受限时仍可以保留结构化诊断覆盖；
3. LLMRCA 说明质量上下文必须进入故障图；
4. AgentChaos 说明任务成功率之外，还要测故障定位率和步骤诊断率；
5. 下一部分把模型网关、RAG、Agent Runtime 和评测组装为完整平台蓝图。

六、综合蓝图与方案分析

用一条 AIOps 请求验证：模型、证据、工具、策略和结果是否真的协同

大模型驱动 AIOps 平台蓝图

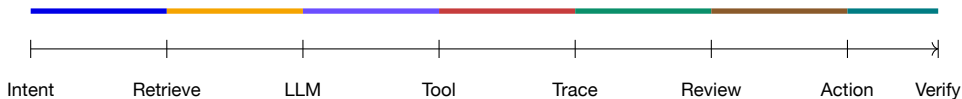


架构重绘的关键不在组件数量，而在“请求如何带着身份、证据、预算和状态穿过每一层”。

综合案例：一次“购物车变慢”的智能运维请求

1. 用户问：“购物车 P99 为什么升高，是否和刚才的发布有关？”
2. Intent Parser 识别服务、时间窗口、变更关系和只读目标；
3. RAG 检索 Runbook、历史事故、服务拓扑和版本约束；
4. Agent 调用指标、Trace、日志、CMDB 和变更工具；
5. 模型生成候选：缓存、连接池、发布配置，并给出支持与反证；
6. 策略阻止自动重启，要求人工确认最小动作；
7. 执行后平台验证业务 SLO、技术信号和副作用，回写 Outcome。

端到端调用链：从用户意图到可验证结果



- ▶ 每段都记录 latency、输入、输出、错误、Token、成本和权限；
- ▶ Trace 在模型调用前建立，在工具和验证完成后关闭；
- ▶ 任何阶段失败，都保存断点和缺口，不能只返回一段错误文本。

统一事件/任务信封

Task Envelope（示例）

`task_id`、`incident_id`、`tenant`、`intent`、`object_scope`、`evidence[]`、`model_route`、`budget`、`tool_calls[]`、`policy_decision`、`outcome`、`human_review`。

- ▶ 第 11 讲的事件 envelope 连接平台对象和状态；
- ▶ 本讲增加模型、检索、工具、预算和 Outcome；
- ▶ 第 13 讲的 RCA Agent 应基于此信封实现，而不是自行定义孤立日志格式。

90 天实施步骤：先观测，再建议，最后受控执行

0-30 天 模型网关、
Trace、Token/成本、RAG
provenance、只读工具。

31-60 天 离线评测、回
放、Shadow Agent、质
量/延迟控制、人工反馈。

61-90 天 审批动作、回
滚验证、预算、故障注入
和阶段验收。

1. 不满足 Trace、权限、评测或回滚门槛时，停在只读/建议模式；
2. 每个阶段保留结果、失败和停止原因；
3. 90 天结束后按 Outcome 决定扩域、重构或停止。

阶段验收：什么条件满足后才能扩大自治？

阶段验收	通过条件	不通过的降级
数据检查 质量检查 运行时门 安全检查 运营门	对象、Trace、RAG provenance 和权限覆盖 任务成功、证据正确、Judge 与人工校准 TTFT/TPOT、Goodput、Token 和成本达标 工具接口说明、审批、回滚、审计和注入测试 采用、反馈、复盘和资产更新持续改善	只读查询、扩大人工范围 低风险任务、模板回答 限流、换模型、异步处理 禁止执行、只给建议 停止扩域、重做数据治理

给一个 AIOps Agent 画架构和状态

1. 选择一个场景：告警摘要、RAG 问答、GPU 巡检或变更评审；
2. 画出模型网关、RAG、工具、策略和 Trace；
3. 定义 8 个任务状态和 3 个停止条件；
4. 写出 2 个离线指标、2 个在线指标和 1 个 Outcome oracle；
5. 说明证据缺失、模型超时、工具失败和人工拒绝时的路径。

10 分钟设计，3 分钟汇报；评分重点是接口、证据和降级，不是模型品牌。

方案评估标准

维度	通过标准	常见失分点
架构分层	网关、运行时、RAG、工具、策略和观测边界清晰	把所有功能塞进一个 Agent
证据格式	有 provenance、版本、权限、时间和缺失字段	只传一段 Prompt
算法结果	有指标、基线、实验条件和结论边界	只写“准确率提升”
安全控制	有预算、审批、回滚、验证和停止条件	直接开放生产执行
使用后的维护	有回放、反馈、成本、采用和版本治理	Demo 结束即交付

回滚推演：只回滚模型，为什么旧行为没有回来？

将一次平台发布记录为版本元组；评测与回放都引用同一发布快照。

发布	模型 / Prompt	索引 / 工具接口	判断
r1	m1 / p1	i1 / t1	历史可用基线
r2	m2 / p2	i2 / t2	新版本
局部回滚	m1 / p2	i2 / t2	从未验收的混合组合
完整配置回滚	m1 / p1	i1 / t1	仍须验证数据与权限条件

回放清单：请求样本、版本元组、检索结果、工具返回、随机性配置与判分规则。

推导与判断 配置回滚只恢复可恢复的版本引用，不能撤销已发生的外部动作；旧索引已删除或权限变化时，也不能声称可以精确重现历史输出。

算例使用假设数据。

下一讲预告：RCA Agent 开发实战

输入 事件、Trace、指标、日志、变更和知识。

推理 假设、工具、证据矩阵、反证和候选排序。

输出 结构化 RCA、置信度、下一步、审批和验证。

第 12 讲回答“平台如何承载模型和 Agent”；第 13 讲把这套接口约定实现成一个 RCA Agent。

参考材料与论文

1. Zhong et al., *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving*, OSDI 2024;
2. Kwon et al., *Efficient Memory Management for Large Language Model Serving with PagedAttention*, SOSP 2023;
3. Huang et al., *Mint: Cost-Efficient Tracing with All Requests Collection via Commonality and Variability Analysis*, ASPLOS 2025;
4. Tan et al., *LLMRCA: Multilevel Root Cause Analysis for LLM Applications Using Multimodal Observability Data*, ACM TOSEM 2025;
5. Tan et al., *AgentChaos: Chaos Engineering for Agent Systems via Programmatic Fault Injection*, ASE 2026 / arXiv:2608.06790;
6. Yao et al., *ReAct*, ICLR 2023; METIS query-level RAG quality-latency materials;
7. OpenTelemetry Semantic Conventions for Generative AI; Langfuse Observability and OTel Integration documentation;
8. 'tooling.pdf'、'agentworkflows.pdf'、'ai-agents.pdf'、'ai-infra-engineer-learning-main'；

论文数字只在对应实验条件下解释；课程案例只用于说明架构和验证方法。

进一步分析

1. 如果模型质量、延迟和成本发生冲突，你会如何设计路由目标？
2. 哪些 RAG 证据必须进入 Trace，哪些内容不应被默认记录？
3. 为什么 Agent 任务成功率高，仍可能有严重的可诊断性问题？
4. 你会把哪项保护检查放在模型之外，为什么？

开放问题 大模型平台的工程成熟度，不由模型回答是否漂亮决定，而由失败是否可解释、可停止、可恢复和可复盘决定。

谢谢

问题、架构蓝图与结果评测讨论